

App Name	BUCKS
The problem	Many young people, particularly students and young adults aged 15–30, struggle to manage their money, track daily expenses, and develop consistent saving habits because traditional finance applications can feel complicated, overwhelming, and unengaging, creating a need for a simple and gamified solo leveling type of financial management experience that motivates users through progression, rewards, challenges, and a virtual companion.
Who is this for	BUCKS is primarily designed for students and young adults aged 15–30 who want a simple, engaging way to manage their allowance, income, expenses, budgets, and savings while staying motivated through gamified progression, rewards, challenges, and Bucks customization. They typically manage their money manually using cash, notes, spreadsheets, calculators, banking/e-wallet apps, or traditional finance apps, while some may not consistently track their spending or savings at all because these methods can feel tedious, complicated, or unmotivating.

Core features (MVP)

#	Feature	Still the MVP?	Flutter pieces it needs	Honest Estimate
1	Login & Registration	Keep	TextField, Form, ElevatedButton, Navigator.push, CircularProgressIndicator, Supabase Auth	5 hrs
2	Home Dashboard	Keep	Scaffold, AppBar, SingleChildScrollView, Column, Row, Card, LinearProgressIndicator, ListView	6 hrs

3	Income & Expense Tracking	Keep	TextField, DropdownButton, DatePicker, Form, ElevatedButton, ListView.builder, Card, model class	8 hrs
4	Transaction History	Keep	ListView.builder, ListTile, Card, PopupMenuButton, showDialog, TextField, Navigator.push	5 hrs
5	Budget Planner	Keep	Form, TextField, DropdownButton, Card, LinearProgressIndicator, ListView.builder, showDialog	8 hrs
6	Savings Goals	Keep	Form, TextField, Card, LinearProgressIndicator, ListView.builder, showDialog, Navigator.push	7hrs
7	Reports & Charts	Keep	Card, Column, Row, ListView, CustomPaint / chart package, model classes	8 hrs
8	Spending Insights	Keep	Card, ListView, LinearProgressIndicator, Icon, conditional widgets, data/model logic	5 hrs
9	Bucks Companion & Reactions	Keep	Image.asset, AnimatedContainer, AnimatedSwitcher, GestureDetector, showDialog, custom model/state logic	8 hrs
10	Daily Missions	Keep	Card, ListView.builder, Checkbox, LinearProgressIndicator, showDialog, mission model	6 hrs
11	XP, Levels & Achievements	Keep	Card, LinearProgressIndicator, GridView.builder, Badge, showDialog, achievement model	7 hrs

12	Bucks Customization	Keep	GridView.builder, GestureDetector, Image.asset, Stack, showModalBottomSheet, customization model	8 hrs
13	Leaderboard	Keep	ListView.builder, Card, CircleAvatar, Image.asset, FutureBuilder, Supabase query, ranking model	7 hrs
14	Bucks Showcase / User Profile	Keep	Stack, Image.asset, Card, GridView.builder, ListView, Navigator.push, profile model	6 hrs
15	Ask Bucks AI Assistant	Stretch	TextField, ListView.builder, Card, CircularProgressIndicator, FutureBuilder, API/service class	12 hrs
16	Profile & Settings	Keep	Scaffold, ListView, ListTile, Switch, showDialog, Navigator.push	4 hrs

Stretch Goals	1. <u>AI Assistant (Ask Bucks)</u> – AI-powered financial advice and personalized spending insights. 2. <u>Advanced Bucks Customization & Social Features</u> – More outfits, accessories, backgrounds, friends, and social interactions. 3. <u>Recurring & Upcoming Transactions</u> – Automatic recurring income/expenses and reminders.
---------------	--

How my app saves data

The question from page 1: if two different people install my app, should they see the same data? (yes / no, and why)

Yes. Different people should have their own personal BUCKS data, but they should be able to see shared data that the app intentionally makes public, such as the

leaderboard. Each user's transactions, budgets, savings goals, and financial information should remain separate.

Roughly how many records does my app hold in a realistic week of use?

Around 50–70 records per active user per week. This includes transactions, budget activity, savings-goal contributions, daily mission progress, achievements, and other user activity.

My choice

Supabase

Why this one and not the others

I chose Supabase because BUCKS is a multi-user app that needs users to save their own financial data while also sharing selected data through features such as the leaderboard. A local database would be simpler, but it would not allow different users to share leaderboard data, so I accepted the tradeoff of having a more complicated cloud setup in exchange for multi-user data and authentication.

What I save, concretely

Tables	Fields
profiles	id, username, level, xp, streak
transactions	id , user_id, type, amount, category, note, date
budget	id, user_id, category, amount, period
savings_goals	id, user_id, name, target_amount, saved_amount, deadline
achievements	id, user_id, achievement_id, unlocked_at
mission_progress	id, user_id, mission_id, date, completed
bucks_profiles	user_id, outfit, hat, accessory, background

The important part is the user_id field. Every user's records are associated with their account, so BUCKS can retrieve only their personal transactions, budgets, and goals.

For the leaderboard, I would only expose safe public fields such as: Username, Level, XP, Streak, Bucks customization, Achievements

I would not expose their actual balance, transaction history, income, or expenses.

Have I tried it yet?

Not yet. I have chosen Supabase based on BUCKS's requirements, but I have not completed the one-hour spike yet. I will do the one-hour Supabase spike before building the full database, starting with creating a user account and saving/retrieving one test record.

I am choosing Supabase because BUCKS needs multi-user data and a shared leaderboard. I accepted the extra setup and learning required for a cloud backend instead of using a simpler local database.

One thing I want to add that the course did not teach

I would like to add charts and AI to BUCKS. I would use the fl_chart package for spending reports and the Gemini API for an AI assistant that can give users budgeting

and spending advice. Charts run on web and can be demonstrated directly in the browser, while if the AI feature does not run on web, I would use a stand-in chat interface for the browser and show the actual AI feature through a phone recording in the final video. Charts are a core feature, while AI is a stretch goal because BUCKS can still function without the AI assistant.

How my project runs when someone else opens it

I am keeping the device_preview wrapper:

Yes, because it lets me preview the app across different device sizes while developing.

My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable:

Not yet, because the app is still being developed and some screens/features still need to be implemented and connected.

Anything that needs real hardware degrades to sample data instead of crashing:

Not applicable, because BUCKS does not currently require hardware features such as GPS, camera, or sensors.

Does anything I plan to build need an API key, a password, or real personal data?

Yes. The Gemini AI feature will require an API key, and Supabase will require project configuration/credentials. These will be kept in a git-ignored .env file locally and repository/deployment secrets when deploying. No real personal financial data will be stored in the public repository.

Data the app remembers

Thing	Fields	Where it is saved
User Profile	id, username, level, xp, streak	Supabase profiles table
Transaction	id, user_id, type, amount, category, note, date	Supabase transactions table
Budget	id, user_id, category, amount,	Supabase budgets table

	period	
Saving Goal	id, user_id, name, target_amount, saved_amount, deadline	Supabase savings_goals table
Achievement	id, user_id, achievement_id, unlocked_at	Supabase achievements table
Mission Progress	id, user_id, mission_id, date, completed	Supabase mission_progress table
Bucks Customization	user_id, outfit, hat, accessory, background	Supabase bucks_profiles table

Screens

Authentication 1. Splash Screen — BUCKS logo + loading 2. Login Screen — username/email + password 3. Register Screen — create account	Main App 1. Home Dashboard — balance, recent transactions, Bucks, XP/level, daily missions 2. Transactions Screen — income/expense list 3. Add Transaction Screen — add income or expense 4. Edit Transaction Screen — modify an existing transaction 5. Transaction Details Screen — view transaction information
Budget & Savings 1. Budget Planner Screen — budgets + progress 2. Create/Edit Budget Screen — set category, amount, and period 3. Savings Goals Screen — list of goals + progress 4. Create/Edit Savings Goal Screen — create or modify a goal 5. Savings Goal Details Screen — contributions and progress	Reports 1. Reports & Charts Screen — spending charts, income vs. expenses 2. Spending Insights Screen — spending breakdown + financial tips
Bucks / Gamification	Social

1. Bucks Companion Screen — Bucks reactions/interactions 2. Daily Missions Screen — 3 daily missions + rewards 3. Achievements Screen — badges and achievements 4. Bucks Customization Screen — outfits, hats, accessories, backgrounds	1. Leaderboard Screen — users ranked by XP 2. Bucks Showcase Screen — public Bucks profile/customization 3. Public User Profile Screen — username, level, XP, achievements, etc.
Account 1. Profile Screen — user's account and progression 2. Settings Screen — notifications, preferences, account settings 3. Edit Profile Screen — username/profile information	Stretch Goal 1. Ask Bucks AI Screen — chat with the Gemini-powered Bucks assistant

Risks, revised**The risk I named last time:**

Learning and implementing Flutter features within the project timeline. It is still a risk, but building M4 and M5 made it smaller because I now have more experience with Flutter widgets, layouts, navigation, and building multiple screens. First step: I will build and connect the core BUCKS screens first, starting September 19, 2026.

A new risk I did not see before:

Integrating Supabase and the Gemini API without exposing credentials or breaking the web version. This is a new risk because BUCKS now depends on cloud data storage and may use AI. First step: I will create a small Supabase test and verify the .env configuration and database connection by September 21, 2026.

What changed and why?

Section	Prelim said	Now says	Why it changed
---------	-------------	----------	----------------

Core features	Included many features, including AI/chat	AI moved to a stretch goal and the core focuses on finance management, gamification, and leaderboard	M4 and M5 showed me that implementing and connecting multiple Flutter screens takes significant time, so I need to prioritize the features essential to BUCKS
Data storage	Local/device storage was considered	Supabase is now the planned database	BUCKS needs separate user accounts and data, while features such as the leaderboard require selected data to be shared between users
Reports	More detailed reports and comparisons	Basic charts and spending insights	I want to keep the reporting useful while reducing the amount of complex data processing and UI needed
AI assistant	AI was considered part of the main feature set	Stretch goal	After considering the additional Gemini API integration, API key management, chat UI, and testing required, I decided BUCKS should work without AI first
Social features	More social interaction was considered	Leaderboard and Bucks showcase only	I want to keep the social aspect manageable instead of adding complex features such as friends and chat

Customization	More extensive Bucks customization	Basic outfits, accessories, and backgrounds	A simpler customization system is more realistic to implement within the project timeline
---------------	------------------------------------	---	---